

INFO145 - Diseño y Análisis de Algoritmos

4 - Cotas Inferiores

Héctor Ferrada
académico

Instituto de Informática
Universidad Austral de Chile

Recordando lo de Cotas Superiores

- Una típica pregunta cuando diseñamos algoritmos es:
Dado un problema $\mathcal{P}$ y una función $f(n)$ ¿Es posible construir un algoritmo $\mathcal{A}$ que resuelva $\mathcal{P}$ en un tiempo de $f(n)$ para toda instancia $\mathcal{I} \in D_{\mathcal{P}}(n)$?

Recordando lo de Cotas Superiores

- Una típica pregunta cuando diseñamos algoritmos es:
Dado un problema $\mathcal{P}$ y una función $f(n)$ ¿Es posible construir un algoritmo $\mathcal{A}$ que resuelva $\mathcal{P}$ en un tiempo de $f(n)$ para toda instancia $\mathcal{I} \in D_{\mathcal{P}}(n)$?
- Suponga que se encuentra un algoritmo $\mathcal{A}$ y su tiempo $f(n)$. Desde cierto punto de vista, este tiempo se puede ver como una **cota superior (o techo)** del peor tiempo posible que toma $\mathcal{A}$ para resolver $\mathcal{P}$, y desde luego, deseamos encontrar $f(n)$ lo más pequeña posible.

Recordando lo de Cotas Superiores

- Una típica pregunta cuando diseñamos algoritmos es:
Dado un problema $\mathcal{P}$ y una función $f(n)$ ¿Es posible construir un algoritmo $\mathcal{A}$ que resuelva $\mathcal{P}$ en un tiempo de $f(n)$ para toda instancia $\mathcal{I} \in D_{\mathcal{P}}(n)$?
- Suponga que se encuentra un algoritmo $\mathcal{A}$ y su tiempo $f(n)$. Desde cierto punto de vista, este tiempo se puede ver como una **cota superior (o techo)** del peor tiempo posible que toma $\mathcal{A}$ para resolver $\mathcal{P}$, y desde luego, deseamos encontrar $f(n)$ lo más pequeña posible.
- Suponga ahora que ha encontrado un nuevo algoritmo $\mathcal{B}$ que resuelve $\mathcal{P}$ con un mejor tiempo $g(n) = o(f(n))$, es decir asintóticamente más pequeña que f .
¿Podemos asegurar ahora que $g(n)$ es el menor costo posible al que podemos aspirar para resolver $\mathcal{P}$, y que por tanto, no hay un costo menor?

Cotas Inferiores

- Ahora nos plantearemos la siguiente pregunta:
Dado un problema $\mathcal{P}$ ¿Cuál es la mayor función $g(n)$, tal que el tiempo de todo algoritmo que resuelva $\mathcal{P}$ sea $\Omega(g(n))$, para cualquier instancia de tamaño n ?

Cotas Inferiores

- Ahora nos plantearemos la siguiente pregunta:
Dado un problema $\mathcal{P}$ ¿Cuál es la mayor función $g(n)$, tal que el tiempo de todo algoritmo que resuelva $\mathcal{P}$ sea $\Omega(g(n))$, para cualquier instancia de tamaño n ?
- Este tiempo se puede ver como una **cota inferior** (o **piso**) para $\mathcal{P}$, y deseamos encontrar $g(n)$ lo más grande posible.
- Las cotas inferiores nos ayudan a entender cuán difícil es un problema de resolver y a evaluar cuan bueno es un algoritmo para resolver el problema.
- Determinar cotas inferiores suele ser más complicado que determinar una cota superior de un algoritmo (su complejidad), ya que ahora debemos asegurar que nuestra función $g(n)$ debe ser un umbral de costo mínimo para cualquier algoritmo que resuelva el problema.

Técnicas para Determinar Cotas Inferiores

- **Estrategia del Peor Adversario.**

Simulamos tener el algoritmo más eficiente ante un terrible adversario (nuestro **peor adversario**) que ordena la entrada para que siempre nuestro algoritmo se enfrente a la peor situación posible.

Técnicas para Determinar Cotas Inferiores

- **Estrategia del Peor Adversario.**

Simulamos tener el algoritmo más eficiente ante un terrible adversario (nuestro **peor adversario**) que ordena la entrada para que siempre nuestro algoritmo se enfrente a la peor situación posible.

- **Teoría de la Información.**

Adaptamos sus estrategias para determinar cotas espaciales, en cuanto al almacenamiento, ahora para establecer cotas inferiores al tiempo requerido para resolver el problema.

Técnicas para Determinar Cotas Inferiores

- **Estrategia del Peor Adversario.**

Simulamos tener el algoritmo más eficiente ante un terrible adversario (nuestro **peor adversario**) que ordena la entrada para que siempre nuestro algoritmo se enfrente a la peor situación posible.

- **Teoría de la Información.**

Adaptamos sus estrategias para determinar cotas espaciales, en cuanto al almacenamiento, ahora para establecer cotas inferiores al tiempo requerido para resolver el problema.

- **Reducción de problemas.**

Se reduce el problema a un problema ya conocido y bien estudiado para utilizar la cota conocida.

I. Estrategia del Peor Adversario

Retomemos el problema de encontrar el mínimo en un arreglo de enteros.

Algoritmo básico $\mathcal{A}$ para la entrada $A[1..n]$:

```
minimo( $A, n$ ) {  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Operaciones primitivas $\rightarrow$ cantidad de comparaciones entre los elementos de A

I. Estrategia del Peor Adversario

Retomemos el problema de encontrar el mínimo en un arreglo de enteros.

Algoritmo básico $\mathcal{A}$ para la entrada $A[1..n]$:

```
minimo( $A, n$ ) {  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Operaciones primitivas $\rightarrow$ cantidad de comparaciones entre los elementos de A

$$\Rightarrow T_{\mathcal{A}}(n) = n - 1$$

I. Estrategia del Peor Adversario

Retomemos el problema de encontrar el mínimo en un arreglo de enteros.

Algoritmo básico $\mathcal{A}$ para la entrada $A[1..n]$:

```
minimo( $A, n$ ) {  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Operaciones primitivas $\rightarrow$ cantidad de comparaciones entre los elementos de A

$\Rightarrow T_{\mathcal{A}}(n) = n - 1 \dots$ pero ¿Es $T_{\mathcal{A}}(n)$ óptimo?

I. Estrategia del Peor Adversario

Retomemos el problema de encontrar el mínimo en un arreglo de enteros.

Algoritmo básico $\mathcal{A}$ para la entrada $A[1..n]$:

```
minimo( $A, n$ ) {  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Operaciones primitivas $\rightarrow$ cantidad de comparaciones **entre los elementos de A**

$\Rightarrow T_{\mathcal{A}}(n) = n - 1 \dots$ pero **¿Es $T_{\mathcal{A}}(n)$ óptimo?** $\dots$ demostraremos que sí:

I. Estrategia del Peor Adversario

Retomemos el problema de encontrar el mínimo en un arreglo de enteros.

Algoritmo básico $\mathcal{A}$ para la entrada $A[1..n]$:

```
minimo( $A, n$ ) {  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Operaciones primitivas $\rightarrow$ cantidad de comparaciones **entre los elementos de A**

$\Rightarrow T_{\mathcal{A}}(n) = n - 1 \dots$ pero **¿Es $T_{\mathcal{A}}(n)$ óptimo?** ... demostraremos que sí:

Suponga que usted propone un nuevo algoritmo $\mathcal{B}$ que realiza

$T_{\mathcal{B}}(n) = n - 2 < T_{\mathcal{A}}(n)$ comparaciones.

I. Estrategia del Peor Adversario

Retomemos el problema de encontrar el mínimo en un arreglo de enteros.

Algoritmo básico $\mathcal{A}$ para la entrada $A[1..n]$:

```
minimo( $A, n$ ) {  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Operaciones primitivas $\rightarrow$ cantidad de comparaciones **entre los elementos de A**

$\Rightarrow T_{\mathcal{A}}(n) = n - 1$... pero **¿Es $T_{\mathcal{A}}(n)$ óptimo?** ... demostraremos que sí:

Suponga que usted propone un nuevo algoritmo $\mathcal{B}$ que realiza

$T_{\mathcal{B}}(n) = n - 2 < T_{\mathcal{A}}(n)$ comparaciones.

Eso quiere decir que hay al menos un elemento de A que no fue comparado,
por ejemplo $A[j]$, $0 < j \leq n$

I. Estrategia del Peor Adversario

Retomemos el problema de encontrar el mínimo en un arreglo de enteros.

Algoritmo básico $\mathcal{A}$ para la entrada $A[1..n]$:

```
minimo( $A, n$ ) {  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Operaciones primitivas $\rightarrow$ cantidad de comparaciones **entre los elementos de A**

$\Rightarrow T_{\mathcal{A}}(n) = n - 1$... pero **¿Es $T_{\mathcal{A}}(n)$ óptimo?** ... demostraremos que sí:

Suponga que usted propone un nuevo algoritmo $\mathcal{B}$ que realiza

$T_{\mathcal{B}}(n) = n - 2 < T_{\mathcal{A}}(n)$ comparaciones.

Eso quiere decir que hay al menos un elemento de A que no fue comparado, por ejemplo $A[j]$, $0 < j \leq n$

Un terrible adversario viene a testear su nuevo algoritmo $\mathcal{B}$, acomodando el arreglo para que justo el menor quede en la posición j .

I. Estrategia del Peor Adversario

Retomemos el problema de encontrar el mínimo en un arreglo de enteros.

Algoritmo básico $\mathcal{A}$ para la entrada $A[1..n]$:

```
minimo( $A, n$ ) {  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Operaciones primitivas $\rightarrow$ cantidad de comparaciones **entre los elementos de A**

$\Rightarrow T_{\mathcal{A}}(n) = n - 1$... pero **¿Es $T_{\mathcal{A}}(n)$ óptimo?** ... demostraremos que sí:

Suponga que usted propone un nuevo algoritmo $\mathcal{B}$ que realiza

$T_{\mathcal{B}}(n) = n - 2 < T_{\mathcal{A}}(n)$ comparaciones.

Eso quiere decir que hay al menos un elemento de A que no fue comparado, por ejemplo $A[j]$, $0 < j \leq n$

Un terrible adversario viene a testear su nuevo algoritmo $\mathcal{B}$, acomodando el arreglo para que justo el menor quede en la posición j .

$\Rightarrow$ **el algoritmo $\mathcal{B}$ es incorrecto**

I. Estrategia del Peor Adversario

Retomemos el problema de encontrar el mínimo en un arreglo de enteros.

Algoritmo básico $\mathcal{A}$ para la entrada $A[1..n]$:

```
minimo( $A, n$ ) {  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
    return  $min$   
}
```

Operaciones primitivas $\rightarrow$ cantidad de comparaciones **entre los elementos de A**

$\Rightarrow T_{\mathcal{A}}(n) = n - 1$... pero **¿Es $T_{\mathcal{A}}(n)$ óptimo?** ... demostraremos que sí:

Suponga que usted propone un nuevo algoritmo $\mathcal{B}$ que realiza

$T_{\mathcal{B}}(n) = n - 2 < T_{\mathcal{A}}(n)$ comparaciones.

Eso quiere decir que hay al menos un elemento de A que no fue comparado, por ejemplo $A[j]$, $0 < j \leq n$

Un terrible adversario viene a testear su nuevo algoritmo $\mathcal{B}$, acomodando el arreglo para que justo el menor quede en la posición j .

$\Rightarrow$ **el algoritmo $\mathcal{B}$ es incorrecto**

$\therefore \mathcal{A}$ es óptimo, y la mayor de las cotas inferiores a $\mathcal{P}$ es precisamente

$T_{\mathcal{A}}(n) = n - 1$.

Encontrar Mínimo y Máximo

$\mathcal{P}$: Encontrar el mínimo y el máximo de un arreglo $X[1..n]$:

Operaciones primitivas $\rightarrow$ las veces que los elementos de X son comparados.

Encontrar Mínimo y Máximo

$\mathcal{P}$: Encontrar el mínimo y el máximo de un arreglo $X[1..n]$:
Operaciones primitivas $\rightarrow$ las veces que los elementos de X son comparados.

¿Qué estrategia usaría para resolver $\mathcal{P}$?

Encontrar Mínimo y Máximo

$\mathcal{P}$: Encontrar el mínimo y el máximo de un arreglo $X[1..n]$:

Operaciones primitivas $\rightarrow$ las veces que los elementos de X son comparados.

¿Qué estrategia usaría para resolver $\mathcal{P}$?

¿Reutilizaría nuestro algoritmo óptimo que encuentra el mínimo?

Encontrar Mínimo y Máximo

$\mathcal{P}$: Encontrar el mínimo y el máximo de un arreglo $X[1..n]$:

Operaciones primitivas $\rightarrow$ las veces que los elementos de X son comparados.

¿Qué estrategia usaría para resolver $\mathcal{P}$?

¿Reutilizaría nuestro algoritmo óptimo que encuentra el mínimo?

¿Cree que esa sea la mejor estrategia?

Encontrar Mínimo y Máximo

$\mathcal{P}$: Encontrar el mínimo y el máximo de un arreglo $X[1..n]$:
Operaciones primitivas $\rightarrow$ las veces que los elementos de X son comparados.

¿Qué estrategia usaría para resolver $\mathcal{P}$?

¿Reutilizaría nuestro algoritmo óptimo que encuentra el mínimo?

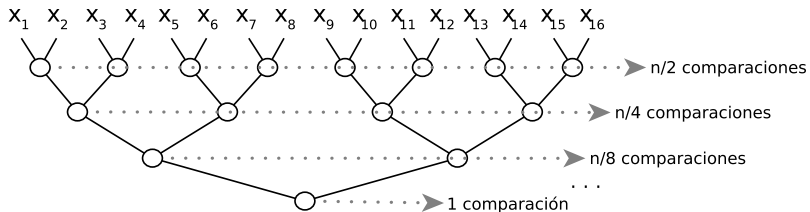
¿Cree que esa sea la mejor estrategia?

Si aplicamos dos veces el algoritmo anterior, primero para el mínimo y luego para el máximo

$\Rightarrow$ Nos daría un costo de $T(n) = 2n - 3$

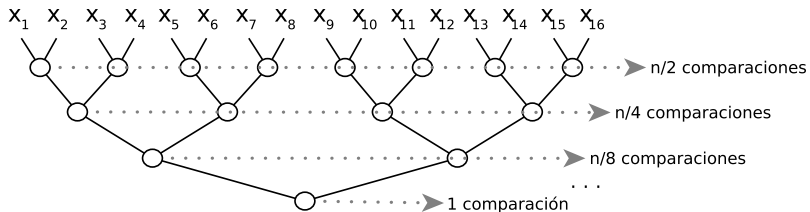
¿Le parece que aplicar dos veces el algoritmo anterior sea estrategia óptima?

Algoritmo del Torneo



La idea es simular la eliminación directa de un torneo (de tenis por ejemplo).

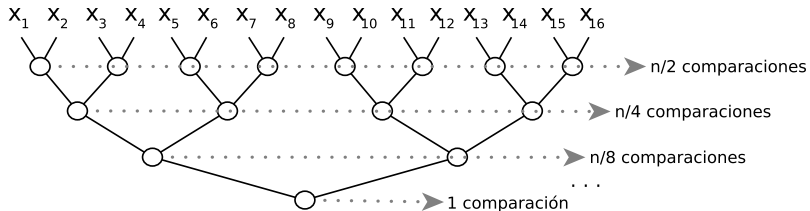
Algoritmo del Torneo



La idea es simular la eliminación directa de un torneo (de tenis por ejemplo). El tiempo $T_1(n)$ para el mínimo es (considere $\lg n = \log_2 n$ y suponga que n es potencia de 2)

$$T_1(n) = \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots + 1$$

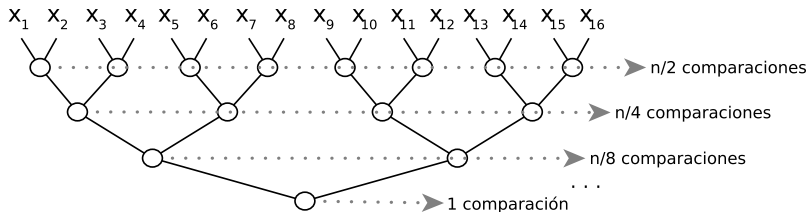
Algoritmo del Torneo



La idea es simular la eliminación directa de un torneo (de tenis por ejemplo). El tiempo $T_1(n)$ para el mínimo es (considere $\lg n = \log_2 n$ y suponga que n es potencia de 2)

$$T_1(n) = \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots + 1 = 1 + 2 + 4 + 8 + \dots + n/2$$

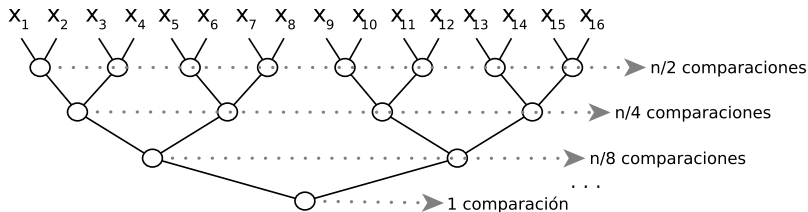
Algoritmo del Torneo



La idea es simular la eliminación directa de un torneo (de tenis por ejemplo). El tiempo $T_1(n)$ para el mínimo es (considere $\lg n = \log_2 n$ y suponga que n es potencia de 2)

$$T_1(n) = \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots + 1 = 1 + 2 + 4 + 8 + \dots + n/2 = \sum_{i=0}^{\lg \frac{n}{2}} 2^i$$

Algoritmo del Torneo

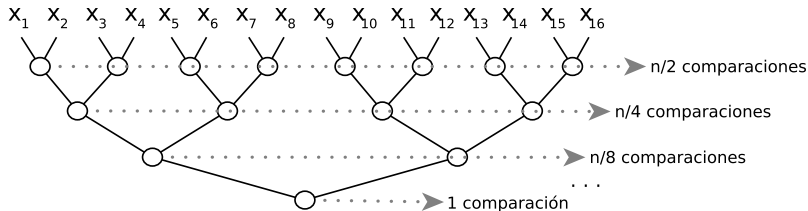


La idea es simular la eliminación directa de un torneo (de tenis por ejemplo). El tiempo $T_1(n)$ para el mínimo es (considere $\lg n = \log_2 n$ y suponga que n es potencia de 2)

$$T_1(n) = \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots + 1 = 1 + 2 + 4 + 8 + \dots + n/2 = \sum_{i=0}^{\lg \frac{n}{2}} 2^i$$

$$\sum_{i=0}^{k-1} a^i = \frac{1 - a^k}{1 - a}$$

Algoritmo del Torneo

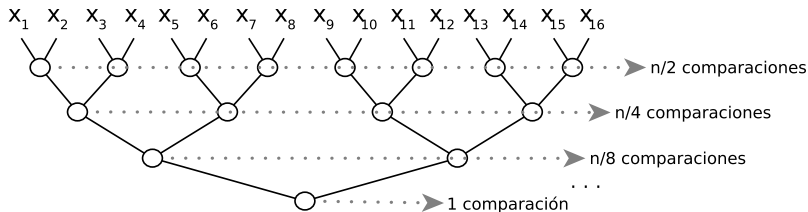


La idea es simular la eliminación directa de un torneo (de tenis por ejemplo). El tiempo $T_1(n)$ para el mínimo es (considere $\lg n = \log_2 n$ y suponga que n es potencia de 2)

$$T_1(n) = \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots + 1 = 1 + 2 + 4 + 8 + \dots + n/2 = \sum_{i=0}^{\lg \frac{n}{2}} 2^i$$

$$\sum_{i=0}^{k-1} a^i = \frac{1-a^k}{1-a} \Rightarrow T_1(n) = \sum_{i=0}^{\lg \frac{n}{2}-1} 2^i + 2^{\lg n/2}$$

Algoritmo del Torneo

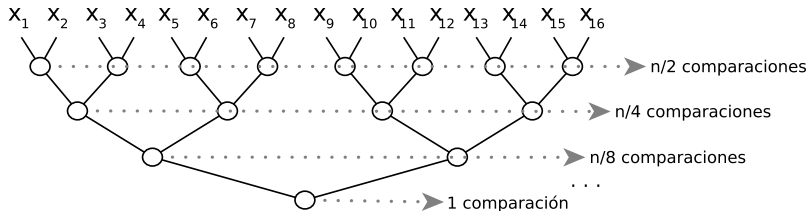


La idea es simular la eliminación directa de un torneo (de tenis por ejemplo). El tiempo $T_1(n)$ para el mínimo es (considere $\lg n = \log_2 n$ y suponga que n es potencia de 2)

$$T_1(n) = \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots + 1 = 1 + 2 + 4 + 8 + \dots + n/2 = \sum_{i=0}^{\lg \frac{n}{2}} 2^i$$

$$\sum_{i=0}^{k-1} a^i = \frac{1-a^k}{1-a} \Rightarrow T_1(n) = \sum_{i=0}^{\lg \frac{n}{2}-1} 2^i + 2^{\lg n/2} = \frac{2^{\lg n/2} - 1}{2 - 1} + \frac{n}{2}$$

Algoritmo del Torneo



La idea es simular la eliminación directa de un torneo (de tenis por ejemplo). El tiempo $T_1(n)$ para el mínimo es (considere $\lg n = \log_2 n$ y suponga que n es potencia de 2)

$$T_1(n) = \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots + 1 = 1 + 2 + 4 + 8 + \dots + n/2 = \sum_{i=0}^{\lg \frac{n}{2}} 2^i$$

$$\sum_{i=0}^{k-1} a^i = \frac{1-a^k}{1-a} \Rightarrow T_1(n) = \sum_{i=0}^{\lg \frac{n}{2}-1} 2^i + 2^{\lg n/2} = \frac{2^{\lg n/2} - 1}{2 - 1} + \frac{n}{2} = \frac{n}{2} - 1 + \frac{n}{2} = n - 1$$

Recuerde que: $b^{\log_b a} = a$, entonces: $\Rightarrow 2^{\lg n/2} = \frac{n}{2}$

Algoritmo del Torneo

- Hemos encontrado el mínimo en $n - 1$ comparaciones $\Rightarrow$ óptimo

Algoritmo del Torneo

- Hemos encontrado el mínimo en $n - 1$ comparaciones $\Rightarrow$ óptimo
- Ahora, la clave será encontrar el máximo solo con los primeros $\lceil \frac{n}{2} \rceil$ perdedores del torneo

Algoritmo del Torneo

- Hemos encontrado el mínimo en $n - 1$ comparaciones $\Rightarrow$ óptimo
- Ahora, la clave será encontrar el máximo solo con los primeros $\lceil \frac{n}{2} \rceil$ perdedores del torneo
- ¿Cuál es el tiempo $T_2(\lceil \frac{n}{2} \rceil)$ para encontrar este máximo?

Algoritmo del Torneo

- Hemos encontrado el mínimo en $n - 1$ comparaciones $\Rightarrow$ óptimo
- Ahora, la clave será encontrar el máximo solo con los primeros $\lceil \frac{n}{2} \rceil$ perdedores del torneo
- ¿Cuál es el tiempo $T_2(\lceil \frac{n}{2} \rceil)$ para encontrar este máximo?
- Aplicamos nuevamente algoritmo del torneo, o el primer algoritmo óptimo, con $\lceil \frac{n}{2} \rceil$ competidores:
 $\Rightarrow T_2(\lceil \frac{n}{2} \rceil) = \lceil \frac{n}{2} \rceil - 1$ comparaciones.

Algoritmo del Torneo

- Hemos encontrado el mínimo en $n - 1$ comparaciones $\Rightarrow$ óptimo
- Ahora, la clave será encontrar el máximo solo con los primeros $\lceil \frac{n}{2} \rceil$ perdedores del torneo
- ¿Cuál es el tiempo $T_2(\lceil \frac{n}{2} \rceil)$ para encontrar este máximo?
- Aplicamos nuevamente algoritmo del torneo, o el primer algoritmo óptimo, con $\lceil \frac{n}{2} \rceil$ competidores:
 $\Rightarrow T_2(\lceil \frac{n}{2} \rceil) = \lceil \frac{n}{2} \rceil - 1$ comparaciones.
- $\therefore T(n) = T_1(n) + T_2(\lceil \frac{n}{2} \rceil) = n + \lceil \frac{n}{2} \rceil - 2 \leq \frac{3n}{2} - 1$

Cota Inferior - Mínimo y Máximo

Suponga la siguiente partición de los n números de la entrada al problema de encontrar el mínimo y máximo en una secuencia de n números:

- A , conjunto con los números que aún no han sido comparados.
- B , conjunto con los números que siempre han ganado sus comparaciones.
- C , conjunto con los números que siempre han perdido sus comparaciones.
- D , conjunto con los números que han ganado y también han perdido sus comparaciones (al menos una vez).

$$\Rightarrow |A| + |B| + |C| + |D| = n$$

Cota Inferior - Mínimo y Máximo

Suponga la siguiente partición de los n números de la entrada al problema de encontrar el mínimo y máximo en una secuencia de n números:

- A , conjunto con los números que aún no han sido comparados.
- B , conjunto con los números que siempre han ganado sus comparaciones.
- C , conjunto con los números que siempre han perdido sus comparaciones.
- D , conjunto con los números que han ganado y también han perdido sus comparaciones (al menos una vez).

$$\Rightarrow |A| + |B| + |C| + |D| = n$$

Al comienzo $(|A|, |B|, |C|, |D|) = (n, 0, 0, 0)$ y un algoritmo correcto debe llegar a $(0, 1, 1, n - 2)$.

Cota Inferior - Mínimo y Máximo

Suponga la siguiente partición de los n números de la entrada al problema de encontrar el mínimo y máximo en una secuencia de n números:

- A , conjunto con los números que aún no han sido comparados.
- B , conjunto con los números que siempre han ganado sus comparaciones.
- C , conjunto con los números que siempre han perdido sus comparaciones.
- D , conjunto con los números que han ganado y también han perdido sus comparaciones (al menos una vez).

$$\Rightarrow |A| + |B| + |C| + |D| = n$$

Al comienzo $(|A|, |B|, |C|, |D|) = (n, 0, 0, 0)$ y un algoritmo correcto debe llegar a $(0, 1, 1, n - 2)$.

Analicemos la siguiente tabla derivada del algoritmo del torneo:

Paso	(A , B , C , D)	Óptimo de Comparaciones
0	$(n, 0, 0, 0)$	0

Cota Inferior - Mínimo y Máximo

Suponga la siguiente partición de los n números de la entrada al problema de encontrar el mínimo y máximo en una secuencia de n números:

- A , conjunto con los números que aún no han sido comparados.
- B , conjunto con los números que siempre han ganado sus comparaciones.
- C , conjunto con los números que siempre han perdido sus comparaciones.
- D , conjunto con los números que han ganado y también han perdido sus comparaciones (al menos una vez).

$$\Rightarrow |A| + |B| + |C| + |D| = n$$

Al comienzo $(|A|, |B|, |C|, |D|) = (n, 0, 0, 0)$ y un algoritmo correcto debe llegar a $(0, 1, 1, n - 2)$.

Analicemos la siguiente tabla derivada del algoritmo del torneo:

Paso	(A , B , C , D)	Óptimo de Comparaciones
0	$(n, 0, 0, 0)$	0
1	$(0, \frac{n}{2}, \frac{n}{2}, 0)$	$n/2$

Cota Inferior - Mínimo y Máximo

Suponga la siguiente partición de los n números de la entrada al problema de encontrar el mínimo y máximo en una secuencia de n números:

- A , conjunto con los números que aún no han sido comparados.
- B , conjunto con los números que siempre han ganado sus comparaciones.
- C , conjunto con los números que siempre han perdido sus comparaciones.
- D , conjunto con los números que han ganado y también han perdido sus comparaciones (al menos una vez).

$$\Rightarrow |A| + |B| + |C| + |D| = n$$

Al comienzo $(|A|, |B|, |C|, |D|) = (n, 0, 0, 0)$ y un algoritmo correcto debe llegar a $(0, 1, 1, n - 2)$.

Analicemos la siguiente tabla derivada del algoritmo del torneo:

Paso	(A , B , C , D)	Óptimo de Comparaciones
0	$(n, 0, 0, 0)$	0
1	$(0, \frac{n}{2}, \frac{n}{2}, 0)$	$n/2$
2	$(0, 1, \frac{n}{2}, \frac{n}{2} - 1)$ $(0, \frac{n}{2}, 1, \frac{n}{2} - 1)$	$\lceil n/2 \rceil - 1$ para los ganadores $\lfloor n/2 \rfloor - 1$ para los perdedores

Cota Inferior - Mínimo y Máximo

Suponga la siguiente partición de los n números de la entrada al problema de encontrar el mínimo y máximo en una secuencia de n números:

- A , conjunto con los números que aún no han sido comparados.
- B , conjunto con los números que siempre han ganado sus comparaciones.
- C , conjunto con los números que siempre han perdido sus comparaciones.
- D , conjunto con los números que han ganado y también han perdido sus comparaciones (al menos una vez).

$$\Rightarrow |A| + |B| + |C| + |D| = n$$

Al comienzo $(|A|, |B|, |C|, |D|) = (n, 0, 0, 0)$ y un algoritmo correcto debe llegar a $(0, 1, 1, n - 2)$.

Analicemos la siguiente tabla derivada del algoritmo del torneo:

Paso	(A , B , C , D)	Óptimo de Comparaciones
0	$(n, 0, 0, 0)$	0
1	$(0, \frac{n}{2}, \frac{n}{2}, 0)$	$n/2$
2	$(0, 1, \frac{n}{2}, \frac{n}{2} - 1)$	$\lceil n/2 \rceil - 1$ para los ganadores
	$(0, \frac{n}{2}, 1, \frac{n}{2} - 1)$	$\lfloor n/2 \rfloor - 1$ para los perdedores
3	$(0, 1, 1, n - 2)$	$T(n) = \frac{n}{2} + \lceil \frac{n}{2} \rceil + \lfloor \frac{n}{2} \rfloor - 2 \leq \frac{3n}{2} - 2$

Cota Inferior - Mínimo y Máximo

Suponga la siguiente partición de los n números de la entrada al problema de encontrar el mínimo y máximo en una secuencia de n números:

- A , conjunto con los números que aún no han sido comparados.
- B , conjunto con los números que siempre han ganado sus comparaciones.
- C , conjunto con los números que siempre han perdido sus comparaciones.
- D , conjunto con los números que han ganado y también han perdido sus comparaciones (al menos una vez).

$$\Rightarrow |A| + |B| + |C| + |D| = n$$

Al comienzo $(|A|, |B|, |C|, |D|) = (n, 0, 0, 0)$ y un algoritmo correcto debe llegar a $(0, 1, 1, n - 2)$.

Analicemos la siguiente tabla derivada del algoritmo del torneo:

Paso	(A , B , C , D)	Óptimo de Comparaciones
0	$(n, 0, 0, 0)$	0
1	$(0, \frac{n}{2}, \frac{n}{2}, 0)$	$n/2$
2	$(0, 1, \frac{n}{2}, \frac{n}{2} - 1)$	$\lceil n/2 \rceil - 1$ para los ganadores
	$(0, \frac{n}{2}, 1, \frac{n}{2} - 1)$	$\lfloor n/2 \rfloor - 1$ para los perdedores
3	$(0, 1, 1, n - 2)$	$T(n) = \frac{n}{2} + \lceil \frac{n}{2} \rceil + \lfloor \frac{n}{2} \rfloor - 2 \leq \frac{3n}{2} - 2$

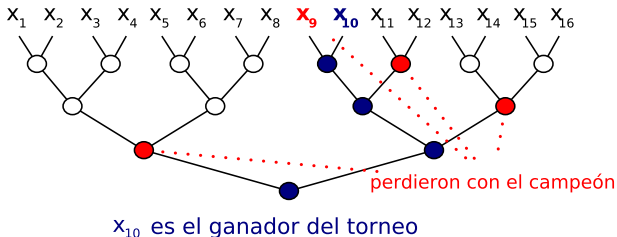
$\therefore$ La cota inferior a este problema es $\frac{3n}{2} - 2$

Encontrar el Mínimo y Segundo Mínimo

Aplicando 2 veces el algoritmo inicial, se resuelve con $2n - 3$ comparaciones. ¿Se puede mejorar este tiempo?

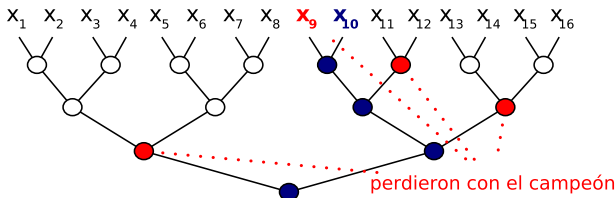
Encontrar el Mínimo y Segundo Mínimo

Aplicando 2 veces el algoritmo inicial, se resuelve con $2n - 3$ comparaciones. ¿Se puede mejorar este tiempo?



Encontrar el Mínimo y Segundo Mínimo

Aplicando 2 veces el algoritmo inicial, se resuelve con $2n - 3$ comparaciones. ¿Se puede mejorar este tiempo?

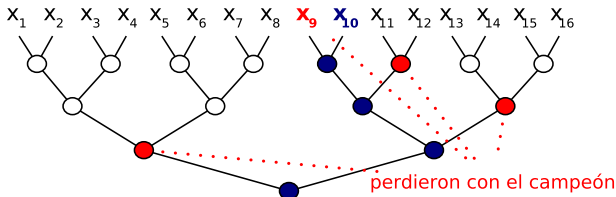


x_{10} es el ganador del torneo

- ¿Cuántos perdieron directamente con el campeón?

Encontrar el Mínimo y Segundo Mínimo

Aplicando 2 veces el algoritmo inicial, se resuelve con $2n - 3$ comparaciones. ¿Se puede mejorar este tiempo?

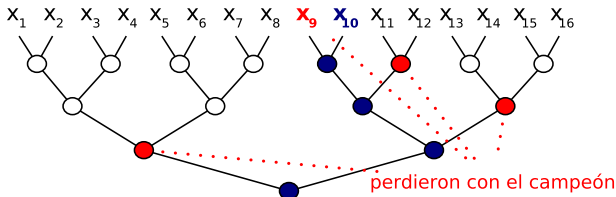


x_{10} es el ganador del torneo

- ¿Cuántos perdieron directamente con el campeón? $\rightarrow \lceil \lg n \rceil$ rivales

Encontrar el Mínimo y Segundo Mínimo

Aplicando 2 veces el algoritmo inicial, se resuelve con $2n - 3$ comparaciones. ¿Se puede mejorar este tiempo?

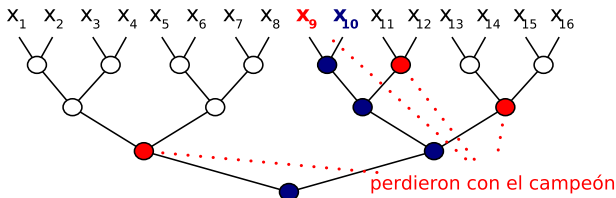


x_{10} es el ganador del torneo

- ¿Cuántos perdieron directamente con el campeón? $\rightarrow \lceil \lg n \rceil$ rivales
- La única posibilidad es que el segundo menor haya sido comparado con el mínimo absoluto (campeón del torneo).

Encontrar el Mínimo y Segundo Mínimo

Aplicando 2 veces el algoritmo inicial, se resuelve con $2n - 3$ comparaciones. ¿Se puede mejorar este tiempo?



x_{10} es el ganador del torneo

- ¿Cuántos perdieron directamente con el campeón? $\rightarrow \lceil \lg n \rceil$ rivales
- La única posibilidad es que el segundo menor haya sido comparado con el mínimo absoluto (campeón del torneo).
- Basta aplicar el algoritmo óptimo del mínimo ahora solo con estos $\lg n$ perdedores, ya que si olvidamos a uno de estos, mi peor adversario puede ubicar al segundo mínimo entre estos:

$$\therefore T(n) = n + \lceil \lg n \rceil - 2 \text{ comparaciones en total}$$

II. Teoría de la Información

- Disciplina encargada de estudiar y determinar el espacio mínimo requerido para representar la información presente en un mensaje.
- Brinda herramientas para determinar la complejidad de un problema, estableciendo cotas inferiores a la cantidad de bits necesarios que debe emitir (escribir) un programa para representar los datos de entrada.

II. Teoría de la Información

- Disciplina encargada de estudiar y determinar el espacio mínimo requerido para representar la información presente en un mensaje.
- Brinda herramientas para determinar la complejidad de un problema, estableciendo cotas inferiores a la cantidad de bits necesarios que debe emitir (escribir) un programa para representar los datos de entrada.
- La idea es reutilizar algunas de sus técnicas para establecer cotas inferiores a la cantidad de operaciones primitivas emitidas por un algoritmo.

Algoritmos basados en Comparaciones Binarias

En teoría de la información podemos reutilizar su principal resultado para este tipo de algoritmos:

- **Para distinguir entre $|U|$ posibilidades (o codificar $|U|$ objetos), necesitas al menos $\log_2 |U|$ bits de información.**
- O dicho de otro modo: Si el conjunto de inputs posibles es $|U|$, entonces no es posible representar a todos sus elementos usando siempre menos de $\log_2 |U|$ bits por elemento.

Algoritmos basados en Comparaciones Binarias

En teoría de la información podemos reutilizar su principal resultado para este tipo de algoritmos:

- **Para distinguir entre $|U|$ posibilidades (o codificar $|U|$ objetos), necesitas al menos $\log_2 |U|$ bits de información.**
- O dicho de otro modo: Si el conjunto de inputs posibles es $|U|$, entonces no es posible representar a todos sus elementos usando siempre menos de $\log_2 |U|$ bits por elemento.

Podemos reutilizar este resultado en problemas que se resuelvan mediante comparaciones binarias, como si cada comparación fuese 1 bit; entonces:

$$WC(\mathcal{P}) = \log_2(\text{todas las salidas posibles a } \mathcal{P})$$

Algoritmos basados en Comparaciones Binarias

Ejemplo. Sea el problema $\mathcal{P}$ de ordenar n elementos (basado en comparaciones binarias).

- Entonces, cada comparación binaria "*entrega*" 1 bit de información:
- $\Rightarrow$ comparar $a \leq b$ su resultado es SI o NO, lo que es exactamente 1 bit en nuestro nuevo modelo
- $\Rightarrow$ un algoritmo basado en comparaciones = "*a una máquina que obtiene bits del input*" y **el algoritmo podría considerarse como un mecanismo de compresión**

Algoritmos basados en Comparaciones Binarias

Ejemplo. Sea el problema $\mathcal{P}$ de ordenar n elementos (basado en comparaciones binarias).

- Entonces, cada comparación binaria "*entrega*" 1 bit de información:
- $\Rightarrow$ comparar $a \leq b$ su resultado es SI o NO, lo que es exactamente 1 bit en nuestro nuevo modelo
- $\Rightarrow$ un algoritmo basado en comparaciones = "*a una máquina que obtiene bits del input*" y el **algoritmo podría considerarse como un mecanismo de compresión**
- ¿Cuanto vale $WC(\mathcal{P})$?

Algoritmos basados en Comparaciones Binarias

Ejemplo. Sea el problema $\mathcal{P}$ de ordenar n elementos (basado en comparaciones binarias).

- Entonces, cada comparación binaria "*entrega*" 1 bit de información:
- $\Rightarrow$ comparar $a \leq b$ su resultado es SI o NO, lo que es exactamente 1 bit en nuestro nuevo modelo
- $\Rightarrow$ un algoritmo basado en comparaciones = "*a una máquina que obtiene bits del input*" y el **algoritmo podría considerarse como un mecanismo de compresión**
- ¿Cuanto vale $WC(\mathcal{P})$?
... es decir ¿De cuántas formas diferentes pueden ordenarse n elementos?

Algoritmos basados en Comparaciones Binarias

Ejemplo. Sea el problema $\mathcal{P}$ de ordenar n elementos (basado en comparaciones binarias).

- Entonces, cada comparación binaria "*entrega*" 1 bit de información:
- $\Rightarrow$ comparar $a \leq b$ su resultado es SI o NO, lo que es exactamente 1 bit en nuestro nuevo modelo
- $\Rightarrow$ un algoritmo basado en comparaciones = "*a una máquina que obtiene bits del input*" y el **algoritmo podría considerarse como un mecanismo de compresión**
- ¿Cuanto vale $WC(\mathcal{P})$?
... es decir ¿De cuántas formas diferentes pueden ordenarse n elementos?
- $WC(\mathcal{P}) = \log(n!)$; ya que existen $n!$ formas distintas de ordenar una colección de n elementos.

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

i) $\log(n!) = \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n)$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\text{i) } \log(n!) = \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n$$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\begin{aligned} \text{i) } \log(n!) &= \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n \\ &\leq \log n + \log n + \log n + \dots + \log n \end{aligned}$$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\begin{aligned} \text{i) } \log(n!) &= \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n \\ &\leq \log n + \log n + \log n + \dots + \log n = n \log n \end{aligned}$$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\begin{aligned} \text{i) } \log(n!) &= \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n \\ &\leq \log n + \log n + \log n + \dots + \log n = n \log n \end{aligned}$$

$$\Rightarrow \log(n!) = O(n \log n)$$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\begin{aligned} \text{i) } \log(n!) &= \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n \\ &\leq \log n + \log n + \log n + \dots + \log n = n \log n \end{aligned}$$

$$\Rightarrow \log(n!) = O(n \log n)$$

ii)

$$\log(n!) = \log 1 + \log 2 + \dots + \log n$$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\begin{aligned} \text{i) } \log(n!) &= \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n \\ &\leq \log n + \log n + \log n + \dots + \log n = n \log n \end{aligned}$$

$$\Rightarrow \log(n!) = O(n \log n)$$

ii)

$$\log(n!) = \log 1 + \log 2 + \dots + \log n = \log 1 + \log 2 + \dots + \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log n$$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\begin{aligned} \text{i) } \log(n!) &= \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n \\ &\leq \log n + \log n + \log n + \dots + \log n = n \log n \end{aligned}$$

$$\Rightarrow \log(n!) = O(n \log n)$$

$$\begin{aligned} \text{ii) } \log(n!) &= \log 1 + \log 2 + \dots + \log n = \log 1 + \log 2 + \dots + \log \frac{n}{2} + \log(\frac{n}{2} + 1) + \dots + \log n \\ &\geq \log \frac{n}{2} + \log(\frac{n}{2} + 1) + \dots + \log(\frac{n}{2} + \frac{n}{2}) \end{aligned}$$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\begin{aligned} \text{i) } \log(n!) &= \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n \\ &\leq \log n + \log n + \log n + \dots + \log n = n \log n \end{aligned}$$

$$\Rightarrow \log(n!) = O(n \log n)$$

$$\begin{aligned} \text{ii) } \log(n!) &= \log 1 + \log 2 + \dots + \log n = \log 1 + \log 2 + \dots + \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log n \\ &\geq \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log\left(\frac{n}{2} + \frac{n}{2}\right) \\ &\geq \log \frac{n}{2} + \log \frac{n}{2} + \dots + \log \frac{n}{2} \end{aligned}$$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\begin{aligned} \text{i) } \log(n!) &= \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n \\ &\leq \log n + \log n + \log n + \dots + \log n = n \log n \end{aligned}$$

$$\Rightarrow \log(n!) = O(n \log n)$$

ii)

$$\begin{aligned} \log(n!) &= \log 1 + \log 2 + \dots + \log n = \log 1 + \log 2 + \dots + \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log n \\ &\geq \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log\left(\frac{n}{2} + \frac{n}{2}\right) \\ &\geq \log \frac{n}{2} + \log \frac{n}{2} + \dots + \log \frac{n}{2} \\ &= \frac{n}{2} \log \frac{n}{2} \end{aligned}$$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\begin{aligned} \text{i) } \log(n!) &= \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n \\ &\leq \log n + \log n + \log n + \dots + \log n = n \log n \end{aligned}$$

$$\Rightarrow \log(n!) = O(n \log n)$$

ii)

$$\begin{aligned} \log(n!) &= \log 1 + \log 2 + \dots + \log n = \log 1 + \log 2 + \dots + \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log n \\ &\geq \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log\left(\frac{n}{2} + \frac{n}{2}\right) \\ &\geq \log \frac{n}{2} + \log \frac{n}{2} + \dots + \log \frac{n}{2} \\ &= \frac{n}{2} \log \frac{n}{2} = \frac{n}{2} \log n - \frac{n}{2} \log 2 \end{aligned}$$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\begin{aligned} \text{i) } \log(n!) &= \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n \\ &\leq \log n + \log n + \log n + \dots + \log n = n \log n \end{aligned}$$

$$\Rightarrow \log(n!) = O(n \log n)$$

$$\begin{aligned} \text{ii) } \log(n!) &= \log 1 + \log 2 + \dots + \log n = \log 1 + \log 2 + \dots + \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log n \\ &\geq \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log\left(\frac{n}{2} + \frac{n}{2}\right) \\ &\geq \log \frac{n}{2} + \log \frac{n}{2} + \dots + \log \frac{n}{2} \\ &= \frac{n}{2} \log \frac{n}{2} = \frac{n}{2} \log n - \frac{n}{2} \log 2 \\ &\geq \frac{1}{2} n \log n \end{aligned}$$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\begin{aligned} \text{i) } \log(n!) &= \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n \\ &\leq \log n + \log n + \log n + \dots + \log n = n \log n \end{aligned}$$

$$\Rightarrow \log(n!) = O(n \log n)$$

ii)

$$\begin{aligned} \log(n!) &= \log 1 + \log 2 + \dots + \log n = \log 1 + \log 2 + \dots + \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log n \\ &\geq \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log\left(\frac{n}{2} + \frac{n}{2}\right) \\ &\geq \log \frac{n}{2} + \log \frac{n}{2} + \dots + \log \frac{n}{2} \\ &= \frac{n}{2} \log \frac{n}{2} = \frac{n}{2} \log n - \frac{n}{2} \log 2 \\ &\geq \frac{1}{2} n \log n = \Omega(n \log n) \end{aligned}$$

Ordenar n elementos

Determinemos el valor de $WC(\mathcal{P}) = \lg(n!)$

$$\begin{aligned} \text{i) } \log(n!) &= \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n \\ &\leq \log n + \log n + \log n + \dots + \log n = n \log n \end{aligned}$$

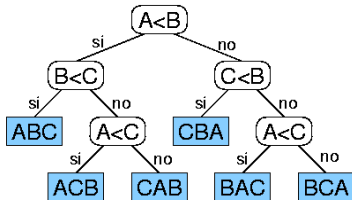
$$\Rightarrow \log(n!) = O(n \log n)$$

$$\begin{aligned} \text{ii) } \log(n!) &= \log 1 + \log 2 + \dots + \log n = \log 1 + \log 2 + \dots + \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log n \\ &\geq \log \frac{n}{2} + \log\left(\frac{n}{2} + 1\right) + \dots + \log\left(\frac{n}{2} + \frac{n}{2}\right) \\ &\geq \log \frac{n}{2} + \log \frac{n}{2} + \dots + \log \frac{n}{2} \\ &= \frac{n}{2} \log \frac{n}{2} = \frac{n}{2} \log n - \frac{n}{2} \log 2 \\ &\geq \frac{1}{2} n \log n = \Omega(n \log n) \end{aligned}$$

$$\therefore \text{ por 1) y ii) } \log(n!) = \Theta(n \log n)$$

Ordenar n elementos

Ejemplo, para ordenar 3 elementos tenemos el siguiente árbol de decisión.



Mínimo necesitamos realizar $\lceil \lg(3!) \rceil = \lceil \lg(6) \rceil = 3$ comparaciones en el peor de los casos para optar a la correctitud de un algoritmo.

III. Reducción de Problemas

Sean $\mathcal{P}$ y Q problemas bien definidos (en nuestro modelo), cuyo costo o Worst Case del problema Q es $W_Q(n) = \Omega(g(n))$.

- En diseño de algoritmos, se suele resolver un **Problema Desconocido** $\mathcal{P}$ reduciendolo a un **Problema Conocido** Q , y así utilizar la respuesta del problema conocido para resolver $\mathcal{P}$.

III. Reducción de Problemas

Sean $\mathcal{P}$ y Q problemas bien definidos (en nuestro modelo), cuyo costo o Worst Case del problema Q es $W_Q(n) = \Omega(g(n))$.

- En diseño de algoritmos, se suele resolver un **Problema Desconocido** $\mathcal{P}$ reduciendolo a un **Problema Conocido** Q , y así utilizar la respuesta del problema conocido para resolver $\mathcal{P}$.
- Ejemplo. Podríamos reducir el problema de encontrar los elementos repetidos de un arreglo, ordenando el arreglo y luego en un recorrido lineal imprimir los números consecutivos que esten repetidos.

III. Reducción de Problemas

Sean $\mathcal{P}$ y Q problemas bien definidos (en nuestro modelo), cuyo costo o Worst Case del problema Q es $W_Q(n) = \Omega(g(n))$.

- En diseño de algoritmos, se suele resolver un **Problema Desconocido** $\mathcal{P}$ reduciendolo a un **Problema Conocido** Q , y así utilizar la respuesta del problema conocido para resolver $\mathcal{P}$.
- Ejemplo. Podríamos reducir el problema de encontrar los elementos repetidos de un arreglo, ordenando el arreglo y luego en un recorrido lineal imprimir los números consecutivos que esten repetidos.
- Así, se cumple la siguiente relación:
 $W_{\mathcal{P}}(n) = R(n) + M \cdot W_Q(n)$, donde M son las veces que fue necesario ejecutar Q y $R(n)$ es el costo de transformar (o reducir) la entrada de $\mathcal{P}(n)$ a $Q(n)$.

III. Reducción de Problemas

Sean $\mathcal{P}$ y Q problemas bien definidos (en nuestro modelo), cuyo costo o Worst Case del problema Q es $W_Q(n) = \Omega(g(n))$.

- En diseño de algoritmos, se suele resolver un **Problema Desconocido** $\mathcal{P}$ reduciendolo a un **Problema Conocido** Q , y así utilizar la respuesta del problema conocido para resolver $\mathcal{P}$.
- Ejemplo. Podríamos reducir el problema de encontrar los elementos repetidos de un arreglo, ordenando el arreglo y luego en un recorrido lineal imprimir los números consecutivos que esten repetidos.
- Así, se cumple la siguiente relación:
 $W_{\mathcal{P}}(n) = R(n) + M \cdot W_Q(n)$, donde M son las veces que fue necesario ejecutar Q y $R(n)$ es el costo de transformar (o reducir) la entrada de $\mathcal{P}(n)$ a $Q(n)$.
- Lo anterior se interpreta como: El costo de $\mathcal{P}$ es igual al costo, $R(n)$, de la transformación (o reducción) más las M veces que fue necesario resolver Q y más el costo de la transformación final de los resultados de Q a la salida esperada en $\mathcal{P}$ (que se espera que sean semejantes a $R(n)$).

III. Cotas Inferiores en Reducción de Problemas

Sean $\mathcal{S}$ y $\mathcal{P}$ problemas bien definidos, donde $\mathcal{S}$ es un problema ya conocido tq: $W_{\mathcal{S}}(n) = \Omega(g(n))$.

A pesar del proceso clásico en la reducción de problemas (slide anterior), para **Determinar Cotas Inferiores** se realiza el proceso inverso:

- Suponga que es posible reducir el problema conocido $\mathcal{S}$ al problema desconocido $\mathcal{P}$ mediante un proceso de reducción que toma un tiempo de $O(g(n))$; es decir, el costo de la reducción es menor o igual al costo de $\mathcal{S}$.

III. Cotas Inferiores en Reducción de Problemas

Sean $\mathcal{S}$ y $\mathcal{P}$ problemas bien definidos, donde $\mathcal{S}$ es un problema ya conocido tq: $W_{\mathcal{S}}(n) = \Omega(g(n))$.

A pesar del proceso clásico en la reducción de problemas (slide anterior), para **Determinar Cotas Inferiores** se realiza el proceso inverso:

- Suponga que es posible reducir el problema conocido $\mathcal{S}$ al problema desconocido $\mathcal{P}$ mediante un proceso de reducción que toma un tiempo de $O(g(n))$; es decir, el costo de la reducción es menor o igual al costo de $\mathcal{S}$.
- También es posible procesar la salida de $\mathcal{P}$ y obtener la respuesta para $\mathcal{S}$ en un tiempo $O(g(n))$.

III. Cotas Inferiores en Reducción de Problemas

Sean $\mathcal{S}$ y $\mathcal{P}$ problemas bien definidos, donde $\mathcal{S}$ es un problema ya conocido tq: $W_{\mathcal{S}}(n) = \Omega(g(n))$.

A pesar del proceso clásico en la reducción de problemas (slide anterior), para **Determinar Cotas Inferiores** se realiza el proceso inverso:

- Suponga que es posible reducir el problema conocido $\mathcal{S}$ al problema desconocido $\mathcal{P}$ mediante un proceso de reducción que toma un tiempo de $O(g(n))$; es decir, el costo de la reducción es menor o igual al costo de $\mathcal{S}$.
- También es posible procesar la salida de $\mathcal{P}$ y obtener la respuesta para $\mathcal{S}$ en un tiempo $O(g(n))$.
- $\Rightarrow W_{\mathcal{S}}(n) \geq O(g(n)) + W_{\mathcal{P}}(n)$

III. Cotas Inferiores en Reducción de Problemas

Sean $\mathcal{S}$ y $\mathcal{P}$ problemas bien definidos, donde $\mathcal{S}$ es un problema ya conocido tq: $W_{\mathcal{S}}(n) = \Omega(g(n))$.

A pesar del proceso clásico en la reducción de problemas (slide anterior), para **Determinar Cotas Inferiores** se realiza el proceso inverso:

- Suponga que es posible reducir el problema conocido $\mathcal{S}$ al problema desconocido $\mathcal{P}$ mediante un proceso de reducción que toma un tiempo de $O(g(n))$; es decir, el costo de la reducción es menor o igual al costo de $\mathcal{S}$.
- También es posible procesar la salida de $\mathcal{P}$ y obtener la respuesta para $\mathcal{S}$ en un tiempo $O(g(n))$.
- $\Rightarrow W_{\mathcal{S}}(n) \geq O(g(n)) + W_{\mathcal{P}}(n) \Rightarrow W_{\mathcal{P}}(n) = \Omega(g(n))$

III. Cotas Inferiores en Reducción de Problemas

Sean $\mathcal{S}$ y $\mathcal{P}$ problemas bien definidos, donde $\mathcal{S}$ es un problema ya conocido tq: $W_{\mathcal{S}}(n) = \Omega(g(n))$.

A pesar del proceso clásico en la reducción de problemas (slide anterior), para **Determinar Cotas Inferiores** se realiza el proceso inverso:

- Suponga que es posible reducir el problema conocido $\mathcal{S}$ al problema desconocido $\mathcal{P}$ mediante un proceso de reducción que toma un tiempo de $O(g(n))$; es decir, el costo de la reducción es menor o igual al costo de $\mathcal{S}$.
- También es posible procesar la salida de $\mathcal{P}$ y obtener la respuesta para $\mathcal{S}$ en un tiempo $O(g(n))$.
- $\Rightarrow W_{\mathcal{S}}(n) \geq O(g(n)) + W_{\mathcal{P}}(n) \Rightarrow W_{\mathcal{P}}(n) = \Omega(g(n))$
- Podemos asegurar lo siguiente: Si $\mathcal{S}$ se resuelve mediante $\mathcal{P}$, entonces $\mathcal{P}$ es a lo menos tan difícil como $\mathcal{S}$

III. Cotas Inferiores en Reducción de Problemas

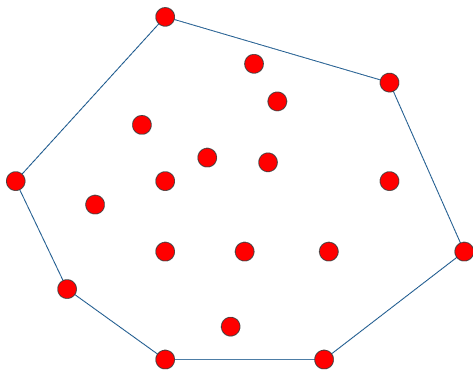
Sean $\mathcal{S}$ y $\mathcal{P}$ problemas bien definidos, donde $\mathcal{S}$ es un problema ya conocido tq: $W_{\mathcal{S}}(n) = \Omega(g(n))$.

A pesar del proceso clásico en la reducción de problemas (slide anterior), para **Determinar Cotas Inferiores** se realiza el proceso inverso:

- Suponga que es posible reducir el problema conocido $\mathcal{S}$ al problema desconocido $\mathcal{P}$ mediante un proceso de reducción que toma un tiempo de $O(g(n))$; es decir, el costo de la reducción es menor o igual al costo de $\mathcal{S}$.
- También es posible procesar la salida de $\mathcal{P}$ y obtener la respuesta para $\mathcal{S}$ en un tiempo $O(g(n))$.
- $\Rightarrow W_{\mathcal{S}}(n) \geq O(g(n)) + W_{\mathcal{P}}(n) \Rightarrow W_{\mathcal{P}}(n) = \Omega(g(n))$
- Podemos asegurar lo siguiente: Si $\mathcal{S}$ se resuelve mediante $\mathcal{P}$, entonces $\mathcal{P}$ es a lo menos tan difícil como $\mathcal{S}$
- $\therefore$ La cota inferior de $\mathcal{S}$ es también cota inferior de $\mathcal{P}$; o dicho de otro modo: El problema desconocido $\mathcal{P}$ no puede ser más fácil que el problema conocido $\mathcal{S}$.

Ejemplo: Convex Hull en 2D

Dado un conjunto S de n puntos en el plano, el Convex hull de S se define como el menor polígono convexo que encierra a los n puntos del conjunto.



Convex Hull Vs. Ordenamiento

- Suponga, que tenemos un algoritmo $\mathcal{A}$ que determina el convex hull en $2D$ en sentido antihorario; por ejemplo el algoritmo de [Graham Scan](#), pero no sabemos cual es la cota inferior del problema del Convex Hull $W_{CHull}(n)$

Convex Hull Vs. Ordenamiento

- Suponga, que tenemos un algoritmo $\mathcal{A}$ que determina el convex hull en $2D$ en sentido antihorario; por ejemplo el algoritmo de [Graham Scan](#), pero no sabemos cual es la cota inferior del problema del Convex Hull $W_{CHull}(n)$
- Tomaremos un problema bien conocido: Ordenamiento, para determinar una cota inferior al Convex Hull.
- Sabemos que el costo de $W_{Sort}(n) = \Omega(n \log n)$

Convex Hull Vs. Ordenamiento

- Suponga, que tenemos un algoritmo $\mathcal{A}$ que determina el convex hull en 2D en sentido antihorario; por ejemplo el algoritmo de [Graham Scan](#), pero no sabemos cual es la cota inferior del problema del Convex Hull $W_{CHull}(n)$
- Tomaremos un problema bien conocido: Ordenamiento, para determinar una cota inferior al Convex Hull.
- Sabemos que el costo de $W_{Sort}(n) = \Omega(n \log n)$

Así, ordenaremos un arreglo numérico $X = \{x_1, x_2, \dots, x_n\}$ mediante el calculo del convex hull de la siguiente manera:

Convex Hull Vs. Ordenamiento

- Suponga, que tenemos un algoritmo $\mathcal{A}$ que determina el convex hull en 2D en sentido antihorario; por ejemplo el algoritmo de [Graham Scan](#), pero no sabemos cual es la cota inferior del problema del Convex Hull $W_{CHull}(n)$
- Tomaremos un problema bien conocido: Ordenamiento, para determinar una cota inferior al Convex Hull.
- Sabemos que el costo de $W_{Sort}(n) = \Omega(n \log n)$

Así, ordenaremos un arreglo numérico $X = \{x_1, x_2, \dots, x_n\}$ mediante el calculo del convex hull de la siguiente manera:

1. Formar la entrada para $\mathcal{A}$, con los puntos $\beta = \{(x_1, x_1^2), (x_2, x_2^2), \dots, (x_n, x_n^2)\}$ a partir de $\alpha = X[1..n]$ en un tiempo de $\rightarrow O(n)$

Convex Hull Vs. Ordenamiento

- Suponga, que tenemos un algoritmo $\mathcal{A}$ que determina el convex hull en 2D en sentido antihorario; por ejemplo el algoritmo de [Graham Scan](#), pero no sabemos cual es la cota inferior del problema del Convex Hull $W_{CHull}(n)$
- Tomaremos un problema bien conocido: Ordenamiento, para determinar una cota inferior al Convex Hull.
- Sabemos que el costo de $W_{Sort}(n) = \Omega(n \log n)$

Así, ordenaremos un arreglo numérico $X = \{x_1, x_2, \dots, x_n\}$ mediante el calculo del convex hull de la siguiente manera:

1. Formar la entrada para $\mathcal{A}$, con los puntos $\beta = \{(x_1, x_1^2), (x_2, x_2^2), \dots, (x_n, x_n^2)\}$ a partir de $\alpha = X[1..n]$ en un tiempo de $\rightarrow O(n)$
2. Ejecutar $\mathcal{A}$ y encontrar el convex hull $H[1..n]$ de β .

Convex Hull Vs. Ordenamiento

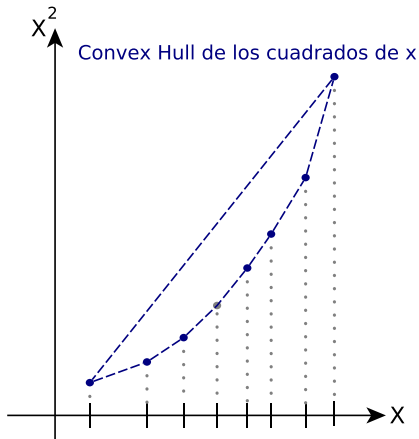
- Suponga, que tenemos un algoritmo $\mathcal{A}$ que determina el convex hull en 2D en sentido antihorario; por ejemplo el algoritmo de [Graham Scan](#), pero no sabemos cual es la cota inferior del problema del Convex Hull $W_{CHull}(n)$
- Tomaremos un problema bien conocido: Ordenamiento, para determinar una cota inferior al Convex Hull.
- Sabemos que el costo de $W_{Sort}(n) = \Omega(n \log n)$

Así, ordenaremos un arreglo numérico $X = \{x_1, x_2, \dots, x_n\}$ mediante el calculo del convex hull de la siguiente manera:

1. Formar la entrada para $\mathcal{A}$, con los puntos $\beta = \{(x_1, x_1^2), (x_2, x_2^2), \dots, (x_n, x_n^2)\}$ a partir de $\alpha = X[1..n]$ en un tiempo de $\rightarrow O(n)$
2. Ejecutar $\mathcal{A}$ y encontrar el convex hull $H[1..n]$ de β .
3. Recorrer $H[1..n]$, en $O(n)$, y obtener los puntos ordenados ascendentemente.

Convex Hull Vs. Ordenamiento

Ordenar un arreglo $X = \{x_1, x_2, \dots, x_n\}$ mediante el cálculo del convex hull en $2D$.



Convex Hull Vs. Ordenamiento

- $\therefore$ Podemos concluir que calcular el Convex Hull en 2D es a lo menos tan difícil como el problema de ordenar una secuencia de numeros

Convex Hull Vs. Ordenamiento

- $\therefore$ Podemos concluir que calcular el Convex Hull en 2D es a lo menos tan difícil como el problema de ordenar una secuencia de numeros
- En cuanto a los costos tenemos:

$$W_{Sort}(n) \geq O(n) + W_{CHull}(n)$$

Convex Hull Vs. Ordenamiento

- $\therefore$ Podemos concluir que calcular el Convex Hull en 2D es a lo menos tan difícil como el problema de ordenar una secuencia de numeros
- En cuanto a los costos tenemos:

$$W_{Sort}(n) \geq O(n) + W_{CHull}(n)$$

$$W_{CHull}(n) \leq \Omega(n \log n) - O(n)$$

Convex Hull Vs. Ordenamiento

- $\therefore$ Podemos concluir que calcular el Convex Hull en 2D es a lo menos tan difícil como el problema de ordenar una secuencia de numeros
- En cuanto a los costos tenemos:

$$W_{Sort}(n) \geq O(n) + W_{CHull}(n)$$

$$W_{CHull}(n) \leq \Omega(n \log n) - O(n)$$

$$W_{CHull}(n) = \Omega(n \log n)$$

Convex Hull Vs. Ordenamiento

- $\therefore$ Podemos concluir que calcular el Convex Hull en 2D es a lo menos tan difícil como el problema de ordenar una secuencia de numeros
- En cuanto a los costos tenemos:

$$W_{Sort}(n) \geq O(n) + W_{CHull}(n)$$

$$W_{CHull}(n) \leq \Omega(n \log n) - O(n)$$

$$W_{CHull}(n) = \Omega(n \log n)$$

- Así podemos acotar el costo del problema de convex hull:

$$W_{CHull}(n) = \Omega(n \log n)$$

Convex Hull Vs. Ordenamiento

- $\therefore$ Podemos concluir que calcular el Convex Hull en 2D es a lo menos tan difícil como el problema de ordenar una secuencia de numeros
- En cuanto a los costos tenemos:

$$W_{Sort}(n) \geq O(n) + W_{CHull}(n)$$

$$W_{CHull}(n) \leq \Omega(n \log n) - O(n)$$

$$W_{CHull}(n) = \Omega(n \log n)$$

- Así podemos acotar el costo del problema de convex hull:

$$W_{CHull}(n) = \Omega(n \log n)$$

¿Asegura esta técnica encontrar la mejor cota al problema?

Extracción en un *heap*

- Pruebe que $\Omega(\lg n)$ comparaciones es cota inferior al problema de extraer el mínimo en un *minheap* —recuerde que el *heap* debe continuar siendo un *minheap* después de la extracción.

Extracción en un *heap*

- Pruebe que $\Omega(\lg n)$ comparaciones es cota inferior al problema de extraer el mínimo en un *minheap* —recuerde que el *heap* debe continuar siendo un *minheap* después de la extracción.

Demostración por Contradicción

- Suponga que extraer el mínimo desde un *minheap*, con n items, en el peor caso se puede hacer en menos que $\lg n$ comparaciones entre sus elementos.

Extracción en un *heap*

- Pruebe que $\Omega(\lg n)$ comparaciones es cota inferior al problema de extraer el mínimo en un *minheap* —recuerde que el *heap* debe continuar siendo un *minheap* después de la extracción.

Demostración por Contradicción

- Suponga que extraer el mínimo desde un *minheap*, con n items, en el peor caso se puede hacer en menos que $\lg n$ comparaciones entre sus elementos.
- $\Rightarrow$ *heapsort* se puede hacer en un tiempo menor a $n \lg n$, ya que equivale a realizar n extracciones de mínimos desde el *minheap*.

Extracción en un *heap*

- Pruebe que $\Omega(\lg n)$ comparaciones es cota inferior al problema de extraer el mínimo en un *minheap* —recuerde que el *heap* debe continuar siendo un *minheap* después de la extracción.

Demostración por Contradicción

- Suponga que extraer el mínimo desde un *minheap*, con n items, en el peor caso se puede hacer en menos que $\lg n$ comparaciones entre sus elementos.
- $\Rightarrow$ *heapsort* se puede hacer en un tiempo menor a $n \lg n$, ya que equivale a realizar n extracciones de mínimos desde el *minheap*.
- $\Rightarrow$ pero *heapsort* es un algoritmo de ordenamiento basado en comparaciones; por tanto no puede resolverse en menos que $n \lg n$ comparaciones en el peor caso.

Extracción en un *heap*

- Pruebe que $\Omega(\lg n)$ comparaciones es cota inferior al problema de extraer el mínimo en un *minheap* —recuerde que el *heap* debe continuar siendo un *minheap* después de la extracción.

Demostración por Contradicción

- Suponga que extraer el mínimo desde un *minheap*, con n items, en el peor caso se puede hacer en menos que $\lg n$ comparaciones entre sus elementos.
- $\Rightarrow$ *heapsort* se puede hacer en un tiempo menor a $n \lg n$, ya que equivale a realizar n extracciones de mínimos desde el *minheap*.
- $\Rightarrow$ pero *heapsort* es un algoritmo de ordenamiento basado en comparaciones; por tanto no puede resolverse en menos que $n \lg n$ comparaciones en el peor caso.
- $\rightarrow \leftarrow$ Contradicción. La suposición inicial no es válida.
- $\Rightarrow \Omega(\lg n)$ es una cota inferior al problema.